

This document is your **single Ctrl+F resource**: Part 1 is the reference card (formulas, steps, tables). Part 2 is every lab exercise type with the question immediately followed by a full worked answer and explanation. Use Ctrl+F to find any term, formula, or exercise type instantly.

NOTE: Part 1 = Reference Card · Part 2 = Exercises with answers directly below each question.

PART 1 — REFERENCE CARD

0 · MODULAR ARITHMETIC TOOLKIT

All crypto in this document uses modular arithmetic — numbers wrap around at a modulus. Three operations you must do by hand:

$a \bmod n$	Divide a by n , take remainder. e.g. $60 \bmod 23 = 14$ because $60 = 2 \times 23 + 14$
$a^k \bmod n$	Repeated squaring. e.g. $5^6 \bmod 23$: $5^2=25 \equiv 2$, $5^4 \equiv 2^2=4$, $5^6=4 \times 2=8 \bmod 23$
$a^{-1} \bmod n$	Find x : $a \times x \equiv 1 \pmod{n}$. Trial: try $x=1,2,3\dots$ e.g. $12^{-1} \bmod 23$: $12 \times 2=24 \equiv 1 \rightarrow \text{answer}=2$

NOTE: Exam shortcut for inverse: just multiply $a \times 1, 2, 3\dots$ until you hit $1 \bmod n$. Always works for small numbers.

1 · RSA — PUBLIC-KEY ENCRYPTION (Lecture 9)

What it is: RSA is a public-key encryption scheme. Anyone can encrypt using the public key; only the owner of the private key can decrypt. Security rests on the **difficulty of factoring** $n = p \times q$: given only n , recovering p and q is computationally infeasible for large primes.

Why it matters: RSA is used in TLS certificates and key transport. **Textbook RSA is deterministic** (same m always gives same c) and is NOT secure in production — real RSA uses **RSA-OAEP** padding to add randomness and prevent structural attacks.

Key generation — step by step:

1	Choose primes	Pick p and q (given in exam). Keep both secret.
2	$n = p \times q$	This is the public modulus.
3	$\phi(n) = (p-1)(q-1)$	Euler totient. Keep secret.
4	Choose e	Public exponent. Must satisfy: $1 < e < \phi(n)$ and $\gcd(e, \phi(n))=1$. Usually given.
5	Find d	$d = e^{-1} \bmod \phi(n)$. Find d so that $e \times d \equiv 1 \pmod{\phi(n)}$. This is the private key.
6	Keys	PUBLIC KEY: (n, e) SECRET KEY: (n, d) — never reveal $d, p, q, \phi(n)$.

Encrypt	$c = m^e \bmod n$
Decrypt	$m = c^d \bmod n$
Verify d	$e \times d \bmod \phi(n)$ must = 1

Lecture example ($p=11, q=13, e=7$):

$n = 11 \times 13$	143
$\phi(n) = 10 \times 12$	120
$d: 7 \times d \equiv 1 \pmod{120}$	$7 \times 103 = 721 = 6 \times 120 + 1 \rightarrow d=103$
Public key	(143, 7)
Secret key	(143, 103)
Encrypt $m=9$	$c = 9^7 \bmod 143 = 48$
Decrypt $c=48$	$m = 48^{103} \bmod 143 = 9 \checkmark$

NOTE: RSA traps: (1) $\phi(n)=(p-1)(q-1)$ NOT $p \times q$. (2) d is the modular inverse of $e \bmod \phi(n)$. (3) Textbook RSA is deterministic — same m always gives same c . (4) Production RSA needs RSA-OAEP padding.

2 · ELGAMAL — PUBLIC-KEY ENCRYPTION (Lecture 9)

What it is: ElGamal is a public-key encryption scheme based on the **discrete logarithm problem**: given $g, p, h = g^x \bmod p$, finding x is hard. Unlike RSA, ElGamal is **probabilistic** — fresh randomness k is chosen per encryption, so the same message encrypts to a different ciphertext each time. Ciphertext is always a **pair (c1, c2)**.

Why it matters: Probabilistic encryption means an attacker cannot detect repeated messages. The encryptor "blinds" m by multiplying with h^k ; the decryptor removes the blinding using the secret x . Reusing k or leaking x breaks security immediately.

Key generation:

1	Choose prime p	e.g. $p=23$. All arithmetic is mod p .
2	Choose generator g	e.g. $g=5$. Powers of g cycle through the group.
3	Choose secret x	Pick any x . This is YOUR SECRET KEY. Never reveal it.
4	Compute h	$h = g^x \bmod p$. e.g. $5^6 \bmod 23 = 8$. Publish h .
5	Public key	(p, g, h) Secret key: x

Encryption (choose fresh k each time):

1	Choose random k	Pick k fresh for each message. Never reuse.
2	$c1 = g^k \bmod p$	e.g. $5^7 \bmod 23 = 17$
3	$c2 = m \times h^k \bmod p$	e.g. $9 \times 8^7 \bmod 23 = 16$
4	Send	Ciphertext is the pair $(c1, c2)$

Decryption:

1	$s = c1^x \bmod p$	Uses your secret x . e.g. $17^6 \bmod 23 = 12$
2	Find $s^{-1} \bmod p$	Find y : $s \times y \equiv 1 \pmod p$. e.g. $12 \times 2 = 24 \equiv 1 \rightarrow s^{-1} = 2$
3	$m = c2 \times s^{-1} \bmod p$	e.g. $16 \times 2 \bmod 23 = 9$ ✓

Key gen	$h = g^x \bmod p$
Encrypt	$c1 = g^k \bmod p$ $c2 = m \cdot h^k \bmod p$
Decrypt	$s = c1^x \bmod p$ $m = c2 \cdot s^{-1} \bmod p$
Why works	$c1^x = (g^k)^x = g^{(kx)} = (g^x)^k = h^k \rightarrow$ dividing $c2$ by h^k recovers m

NOTE: ElGamal traps: (1) $h=g^x$ — x is secret, h is public. (2) k is fresh EVERY encryption. (3) $c2 = m \times h^k$ (multiply by h^k not just h). (4) Decryption: compute s first, then find s^{-1} , then $m=c2 \times s^{-1}$. (5) If k is reused for two messages, the ratio $m1/m2$ is leaked.

RSA vs ElGamal — Comparison Table

Property	RSA	ElGamal
Hard problem	Integer factorisation	Discrete logarithm
Public key	(n, e)	(p, g, h)
Secret key	d	x
Encrypt	$c = m^e \bmod n$ (1 value)	$c1=g^k \bmod p$; $c2=m \cdot h^k \bmod p$ (pair)
Decrypt	$m = c^d \bmod n$	$s=c1^x \bmod p$; $m=c2 \cdot s^{-1} \bmod p$
Randomness	None (deterministic)	Fresh k every time (probabilistic)
Production fix	RSA-OAEP padding	Already probabilistic; use large p

3 · MPC, BOOLEAN SECRET SHARING & BEAVER TRIPLES (Lecture 10)

What it is: Multi-Party Computation (MPC) lets multiple parties compute a function on their **private inputs** while revealing **only the agreed output**. Example: Alice and Bob check mutual interest (a AND b) without revealing individual preference if the answer is 0. Security relies on **secret sharing**: split inputs into "shares" so no single party sees raw data.

Why it matters: MPC enables privacy-preserving computation — salary comparison, genome matching, fraud detection — where raw data cannot be shared. **XOR operations are free** (local, no communication). **AND operations cost** — they require preprocessed **Beaver triples**.

Boolean Secret Sharing:

1	Split secret a	Choose random bit r . $\text{Share_A} = a \text{ XOR } r$. $\text{Share_B} = r$.
2	Reconstruct a	$\text{Share_A XOR Share_B} = (a \text{ XOR } r) \text{ XOR } r = a$. Just XOR the two shares.
3	Why secure?	Each share alone is uniformly random in $\{0,1\}$ — reveals nothing about a .
4	XOR is FREE	$c = a \text{ XOR } b$: each party XORs their own shares locally. Zero communication.
5	AND is COSTLY	Local AND of shares does NOT reconstruct $a \text{ AND } b$. Need Beaver triple.

XOR Sharing Reconstruction Table:

Secret a	Random r	Share_A = a XOR r	Share_B = r	Reconstruct: A XOR B
0	0	0	0	$0 \text{ XOR } 0 = 0 \checkmark$
0	1	1	1	$1 \text{ XOR } 1 = 0 \checkmark$
1	0	1	0	$1 \text{ XOR } 0 = 1 \checkmark$
1	1	0	1	$0 \text{ XOR } 1 = 1 \checkmark$

Beaver Triples — AND Gate:

A Beaver triple is a precomputed tuple (i, j, k) of secret-shared random bits where $k = i \text{ AND } j$. Generated offline before real inputs arrive. Allows AND to be computed with just two reveals (d and e) — never exposing a or b directly.

1	Offline: get triple	Random bits i, j, k where $k = i \text{ AND } j$. All secret-shared.
2	Compute masks	$d = a \text{ XOR } i$ $e = b \text{ XOR } j$
3	Open d and e	Both parties learn d and e . SAFE because i, j are random masks (OTP logic).
4	Compute a AND b	$a \text{ AND } b = k \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (d \text{ AND } e)$
5	Why only one adds d AND e	Only one party adds $(d \text{ AND } e)$ to their k share — XOR reconstruction still correct.

AND Formula — proved from scratch:

Start	$a = d \text{ XOR } i$ $b = e \text{ XOR } j$
Expand	$a \text{ AND } b = (d \text{ XOR } i) \text{ AND } (e \text{ XOR } j)$
Distribute	$= (d \text{ AND } e) \text{ XOR } (d \text{ AND } j) \text{ XOR } (i \text{ AND } e) \text{ XOR } (i \text{ AND } j)$
$k = i \text{ AND } j$	$= k \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (d \text{ AND } e) \leftarrow \text{THE FORMULA}$

AND Formula Quick-Reference Drill Table:

a	b	i	j	k	$d=a \wedge i$	$e=b \wedge j$	$d \& j$	$e \& i$	$d \& e$	RESULT	check
0	0	0	0	0	0	0	0	0	0	0	0✓
0	1	0	1	0	0	0	0	0	0	0	0✓
1	0	1	0	0	0	0	0	0	0	0	0✓
1	1	1	1	1	0	0	0	0	0	1	1✓
1	1	0	0	0	1	1	0	0	1	1	1✓
1	0	0	1	0	1	1	1	0	1	0	0✓
0	1	1	0	0	1	1	0	1	1	0	0✓
0	0	1	1	1	1	1	1	1	1	0	0✓

NOTE: MPC traps: (1) XOR just XOR the shares — no Beaver needed. (2) AND needs the Beaver formula — not just ANDing shares. (3) $d=a \text{ XOR } i$ is safe to reveal because i is random (OTP logic). (4) Reusing a triple for two computations leaks XOR of inputs. (5) Security fails if both servers collude (they reconstruct the secret).

Caesar cipher	Shift each letter by $k \bmod 26$. Decrypt: shift back by k . 26 possible keys – try all.
Substitution	Each letter → unique different letter (not fixed shift). Break by frequency analysis. E is most common (~12.7%).
DES	64-bit block · 56-bit key (8 parity) · 16 rounds · BROKEN – key too short.
AES-128	128-bit block · 128-bit key · 10 rounds · FIPS PUB 197 · current standard.
AES-256	128-bit block · 256-bit key · 14 rounds · highest AES security.
OTP	Key = length of plaintext · never reuse · XOR-based · perfect secrecy · impractical.
ChaCha20	Modern stream cipher + MAC · used in TLS 1.3 · fast on mobile.
Brute force	n -bit key at 2^r keys/s: time = $2^{(n-r)}$ seconds. 80-bit at $2^{30}/s = 2^{50}s \approx 35.7M$ years.

TLS Cipher Suite — how to read it:

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256	TLS 1.2 example
ECDHE	Key exchange (Elliptic Curve Diffie-Hellman Ephemeral) → forward secrecy
RSA	Authentication – server proves identity via certificate signature (NOT encryption)
AES_128_GCM	Symmetric AEAD cipher – encrypts all data after handshake
SHA256	Hash function – used in key derivation and handshake integrity
TLS 1.3 example: TLS_AES_128_GCM_SHA256	Key exchange is implicit (always ECDHE). Only cipher + hash listed.

PART 2 — EXERCISES WITH WORKED ANSWERS

Every question below is followed immediately by its full worked answer and explanation. These mirror every exercise type from Lab 09 and Lab 10 with fresh parameters.

LAB 09 · CRYPTOGRAPHY

Exercise 1 — Classical Ciphers

Q1 — Caesar Cipher — Decrypt (key unknown)

The following ciphertext was encrypted with a Caesar cipher. The key is NOT 3.

Find the correct shift and state the plaintext.

Ciphertext: **PELCGBTENCUL VF SHA**

ANSWER — Q1

Method: Try all 26 shifts until readable English appears.

Key = 13 (ROT-13 — each letter shifted forward 13, wraps at Z):

$P(15)-13=2=C$ | $E(4)-13=17=R$ | $L(11)-13=24=Y$ | $C(2)-13=15=P$ | $G(6)-13=19=T$ | $O(14)-13=1=B$ |

$T(19)-13=6=G$ | $E(4)-13=17=R$ | $N(13)-13=0=A$ | $C(2)-13=15=P$ | $U(20)-13=7=H$ | $L(11)-13=24=Y$

$V(21)-13=8=I$ | $F(5)-13=18=S$ | $S(18)-13=5=F$ | $H(7)-13=20=U$ | $N(13)-13=0=A$

Plaintext: CRYPTOGRAPHY IS FUN

Explanation: Caesar cipher has only 26 possible keys. To decrypt: for each shift k from 1 to 25, subtract k from every letter (mod 26) and check if the result is English. With $k=13$ the output is readable. The Caesar cipher is trivially broken because the key space is tiny.

Q2 — Caesar Cipher — Encrypt (key given)

Using key $k = 8$, encrypt the following plaintext.

Show the shift for each letter.

Plaintext: **NETWORK SECURITY**

ANSWER — Q2

Rule: For each letter, add 8 to its position ($A=0$, $B=1$, ..., $Z=25$), then mod 26.

$N(13)+8=21=V$ | $E(4)+8=12=M$ | $T(19)+8=27 \bmod 26=1=B$ | $W(22)+8=30 \bmod 26=4=E$ | $O(14)+8=22=W$ | $R(17)+8=25=Z$ |
 $K(10)+8=18=S$

$S(18)+8=26 \bmod 26=0=A$ | $E(4)+8=12=M$ | $C(2)+8=10=K$ | $U(20)+8=28 \bmod 26=2=C$ | $R(17)+8=25=Z$ | $I(8)+8=16=Q$ |
 $T(19)+8=27=1=B$ | $Y(24)+8=32 \bmod 26=6=G$

Ciphertext: VMBEWZS AMKCZQBG

Explanation: To encrypt with Caesar, add k to each letter position mod 26. When the sum exceeds 25, wrap around (mod 26). To decrypt: subtract k . The cipher is symmetric — encrypt and decrypt use the same key.

Q3 — Substitution Cipher — Breaking Method

You intercept a long ciphertext encrypted with a monoalphabetic substitution cipher (each plaintext letter maps to a fixed but arbitrary ciphertext letter).

- (a) What technique do you use to break it? Explain the process.
- (b) Which English letter is most common, and how does that help?
- (c) Why does this attack NOT work on the One-Time Pad?

ANSWER — Q3

(a) Frequency analysis:

1. Count how often each ciphertext letter appears.
2. Map the most frequent ciphertext letter to E (most common in English, ~12.7%).
3. Map the next most frequent to T (~9.1%), then A (~8.2%), O, I, N, S, H, R...
4. Use common bigrams (TH, HE, IN, ER) and trigrams (THE, AND, ING) to refine.
5. Use context and guesswork to fill remaining letters.

(b) E is the most common letter in English (~12.7%). The most frequent ciphertext letter almost certainly maps to E, giving the first substitution entry immediately.

(c) OTP is immune because the key is as long as the message and used only once — each plaintext letter is XORed with a DIFFERENT random key bit. Letter frequencies in the plaintext are completely destroyed. The ciphertext is perfectly random and statistically independent of the plaintext.

Exercise 2 — Argue and Explain

Q4 — AES vs RSA — Which Is Faster and Why?

- (a) Explain why AES is faster than RSA.
- (b) TLS uses both. What does each one do, and why use both?
- (c) If AES is faster, why not use it for everything including key exchange?

ANSWER — Q4

(a) AES is orders of magnitude faster than RSA because:

- AES operates on fixed 128-bit blocks using simple substitution-permutation operations.
- Modern CPUs have AES-NI hardware instructions — AES is essentially "free" in silicon.
- RSA requires modular exponentiation with 2048-bit integers — thousands of large multiplications per operation.
- Rule of thumb: AES encrypts gigabytes per second; RSA encrypts kilobytes per second.

(b) In TLS:

- ECDHE/RSA: used ONCE at the start of the handshake to authenticate the server and establish a shared session key. Asymmetric crypto solves the key distribution problem.
- AES: used for ALL actual data transfer using the established session key. Fast symmetric crypto.
- This hybrid approach: security of asymmetric (no pre-shared secret needed) + speed of symmetric.

(c) AES requires a shared secret key to already exist — it cannot create one from scratch between two strangers. RSA/ECDHE solves the key distribution problem: two parties who have never met can agree on a secret over an untrusted channel. AES then uses that secret efficiently.

Q5 — Why AES Has Not Been Broken

- (a) What would it take to break AES-128 by brute force? Give a calculation.
- (b) Name two structural properties that have kept AES secure since 2001.
- (c) Does using AES-256 instead of AES-128 mean double the security?

ANSWER — Q5

(a) Brute force calculation for AES-128:

- Key space: $2^{128} \approx 3.4 \times 10^{38}$ keys.
- Even at 2^{50} keys/second (far beyond any current hardware): $2^{128} / 2^{50} = 2^{78}$ seconds.
- 2^{78} seconds $\approx 9.5 \times 10^{15}$ years — roughly a million times the age of the universe.
- AES-128 is computationally infeasible to brute-force for any foreseeable future.

(b) Two structural properties:

- **No known structural weakness:** decades of cryptanalysis by global experts found no shortcut attack faster than brute force against the full cipher.
- **Good diffusion and confusion:** the SubBytes, ShiftRows, MixColumns, and AddRoundKey steps ensure each output bit depends on all input bits in a complex, non-linear way.

(c) No — not double. AES-256 has 2^{128} MORE keys than AES-128 (not double — exponential). In practice, both are considered equally unbreakable by classical computers. AES-256 is preferred for post-quantum resistance since Grover's algorithm halves the effective key length, making 256-bit $\rightarrow$ 128-bit equivalent security.

Q6 — RSA vs ElGamal — Write a Comparison

Write 5–6 sentences comparing RSA and ElGamal. Cover:

- The mathematical hard problem each relies on
- Deterministic vs probabilistic encryption
- Ciphertext structure
- One practical advantage of ElGamal over textbook RSA

ANSWER — Q6

Model answer:

RSA is based on the difficulty of **integer factorisation**: given $n = p \times q$ where p and q are large primes, recovering p and q is computationally infeasible. ElGamal is based on the **discrete logarithm problem**: given g , p , and $h = g^x \bmod p$, finding x is hard.

Textbook RSA is **deterministic** — the same plaintext always produces the same ciphertext. This leaks information about repeated messages and is vulnerable to chosen-plaintext attacks. ElGamal is **probabilistic** — a fresh random value k is chosen for each encryption, so the same plaintext produces a different ciphertext every time. This property is called semantic security (IND-CPA security).

RSA produces a **single ciphertext value** c . ElGamal produces a **pair** (c_1, c_2) because the encryptor must embed the randomness k in $c_1 = g^k \bmod p$.

A key practical advantage of ElGamal: it is probabilistic **by construction**, without requiring additional padding. Textbook RSA needs RSA-OAEP padding to achieve the same semantic security property — without it, RSA is dangerously insecure in production.

Exercise 3 — RSA Calculation

Q7 — RSA — Full Worked Calculation (p=7, q=11, e=13)

(a) Compute n and phi(n).

(b) Find private exponent d. Show the verification.

(c) State the public and secret keys.

(d) Encrypt m = 3.

(e) Decrypt your ciphertext. Verify you recover m = 3.

(f) Why must phi(n) be kept secret?

ANSWER — Q7

(a) $n = 7 \times 11 = 77$ | $\phi(n) = (7-1)(11-1) = 6 \times 10 = 60$

(b) Find d: $13 \times d \equiv 1 \pmod{60}$.

Try: $13 \times 1 = 13$, $13 \times 2 = 26$, $13 \times 3 = 39$, $13 \times 4 = 52$, $13 \times 5 = 65 \equiv 5$, $13 \times 6 = 78 \equiv 18$, ...

$13 \times 37 = 481$. Check: $481 = 8 \times 60 + 1 \rightarrow \mathbf{d = 37}$.

Verify: $13 \times 37 = 481$. $481 \bmod 60 = 481 - 8 \times 60 = 481 - 480 = 1 \checkmark$

(c) Public key: **(77, 13)** | Secret key: **(77, 37)**

(d) $c = 3^{13} \bmod 77$.

$3^2 = 9$, $3^4 = 81 \bmod 77 = 4$, $3^8 = 4^2 = 16$, $3^{13} = 3^8 \times 3^4 \times 3^1 = 16 \times 4 \times 3 = 192 \bmod 77$.

$192 = 2 \times 77 + 38 \rightarrow \mathbf{c = 38}$

(e) $m = 38^{37} \bmod 77$.

$38^2 = 1444 \bmod 77$: $1444 = 18 \times 77 + 58 \rightarrow 58$.

$38^4 = 58^2 = 3364 \bmod 77$: $3364 = 43 \times 77 + 53 \rightarrow 53$.

$38^8 = 53^2 = 2809 \bmod 77$: $2809 = 36 \times 77 + 37 \rightarrow 37$.

$38^{16} = 37^2 = 1369 \bmod 77$: $1369 = 17 \times 77 + 60 \rightarrow 60$.

$38^{32} = 60^2 = 3600 \bmod 77$: $3600 = 46 \times 77 + 58 \rightarrow 58$.

$37 = 32 + 4 + 1$: $38^{37} = 38^{32} \times 38^4 \times 38^1 = 58 \times 53 \times 38 \bmod 77$.

$58 \times 53 = 3074 \bmod 77$: $3074 = 39 \times 77 + 71 \rightarrow 71$. $71 \times 38 = 2698 \bmod 77$: $2698 = 35 \times 77 + 2695 \rightarrow \mathbf{3} \checkmark$

(f) $\phi(n) = (p-1)(q-1)$. Knowing $\phi(n)$ lets anyone compute $d = e^{-1} \bmod \phi(n)$, which is the private key. Protecting $\phi(n)$ is equivalent to protecting d.

Q8 — RSA — Second Calculation (p=5, q=13, e=11)

(a) Find n , $\phi(n)$, d .

(b) Encrypt $m = 4$.

(c) Why is it crucial that p and q are large in real RSA?

ANSWER — Q8

(a) $n = 5 \times 13 = 65$ | $\phi(n) = 4 \times 12 = 48$

Find d : $11 \times d \equiv 1 \pmod{48}$.

$11 \times 1 = 11$, $11 \times 2 = 22$, ..., $11 \times 22 = 242 = 5 \times 48 + 2$, ...

Faster: use extended Euclidean or trial: $11 \times 35 = 385 = 8 \times 48 + 1 \rightarrow d = 35$.

Verify: $11 \times 35 = 385$. $385 \bmod 48$: $385 = 8 \times 48 + 1 \rightarrow 1 \checkmark$

(b) $c = 4^{11} \bmod 65$.

$4^2 = 16$, $4^4 = 256 \bmod 65 = 256 - 3 \times 65 = 256 - 195 = 61$, $4^8 = 61^2 \bmod 65 = 3721 \bmod 65 = 3721 - 57 \times 65 = 3721 - 3705 = 16$.

$4^{11} = 4^8 \times 4^2 \times 4^1 = 16 \times 16 \times 4 \bmod 65 = 256 \bmod 65 \times 4 = 61 \times 4 = 244 \bmod 65$.

$244 = 3 \times 65 + 49 \rightarrow c = 49$.

(c) In real RSA, p and q are ~ 1024 -bit primes each, making $n \sim 2048$ bits. The security of RSA depends on the difficulty of factoring n back into p and q . For small primes (like our toy examples), n can be factored instantly. For 2048-bit n , no known classical algorithm can factor it in polynomial time — the best known general factoring algorithms (GNFS) would take longer than the age of the universe.

Q9 — RSA — Conceptual: Textbook vs Production Security

(a) Give a concrete attack against textbook RSA that OAEP prevents.

(b) What exactly does RSA-OAEP add to prevent that attack?

(c) An attacker has n and e . What hard problem must they solve to get d ?

ANSWER — Q9

(a) Chosen plaintext attack (determinism attack):

Because textbook RSA is deterministic, an attacker who can observe ciphertexts can build a dictionary: compute $c = m^e \bmod n$ for all small m values and look up observed ciphertexts. Also: textbook RSA is "malleable" — given ciphertext c of m , compute $c' = (2^e \times c) \bmod n$ which is the encryption of $2m$. The attacker can manipulate ciphertexts without knowing m .

(b) RSA-OAEP (Optimal Asymmetric Encryption Padding):

Before RSA encryption, the message is combined with a random seed using two hash functions (MGF). The padded message looks random and different every time, even for the same plaintext. This eliminates the dictionary attack and the malleability property.

(c) To recover d from (n, e) , the attacker must:

1. Factor $n = p \times q$ (integer factorisation — hard for large n)

2. Compute $\phi(n) = (p-1)(q-1)$

3. Compute $d = e^{-1} \bmod \phi(n)$

Without factoring n , computing $\phi(n)$ is believed to be equally hard. RSA security reduces to the hardness of integer factorisation.

Exercise 4 — ElGamal Calculation

Q10 — ElGamal — Full Calculation (p=19, g=2, x=5, m=7, k=3)

- (a) Compute public value h.
- (b) State the public and secret keys.
- (c) Encrypt m=7 with randomness k=3. Compute (c1, c2).
- (d) Decrypt (c1, c2). Show every step.
- (e) Verify you recover m=7.
- (f) Encrypt m=7 again with k=6. Do you get the same ciphertext?

ANSWER — Q10

(a) $h = g^x \bmod p = 2^5 \bmod 19 = 32 \bmod 19 = 13$

(b) Public key: **(19, 2, 13)** | Secret key: **x = 5**

(c) Encrypt:

$$c1 = g^k \bmod p = 2^3 \bmod 19 = 8$$

$$h^k = 13^3 \bmod 19: 13^2 = 169 \bmod 19 = 169 - 8 \times 19 = 169 - 152 = 17. \quad 13^3 = 17 \times 13 = 221 \bmod 19 = 221 - 11 \times 19 = 221 - 209 = 12.$$

$$c2 = m \times h^k \bmod p = 7 \times 12 \bmod 19 = 84 \bmod 19 = 84 - 4 \times 19 = 84 - 76 = 8$$

Ciphertext: (c1, c2) = (8, 8)

(d) Decrypt:

$$s = c1^x \bmod p = 8^5 \bmod 19.$$

$$8^2 = 64 \bmod 19 = 64 - 3 \times 19 = 7. \quad 8^4 = 7^2 = 49 \bmod 19 = 49 - 2 \times 19 = 11. \quad 8^5 = 11 \times 8 = 88 \bmod 19 = 88 - 4 \times 19 = 12.$$

$$s = 12.$$

Find $s^{-1} \bmod 19$: need y such that $12 \times y \equiv 1 \pmod{19}$.

$$12 \times 8 = 96. \quad 96 \bmod 19 = 96 - 5 \times 19 = 96 - 95 = 1 \rightarrow s^{-1} = 8.$$

$$m = c2 \times s^{-1} \bmod p = 8 \times 8 \bmod 19 = 64 \bmod 19 = 64 - 3 \times 19 = 7.$$

(e) m = 7 ✓

(f) With k=6:

$$c1 = 2^6 \bmod 19 = 64 \bmod 19 = 7. \text{ (Different from } c1=8 \text{ with } k=3.)$$

$$h^6 = 13^6 \bmod 19: 13^2 = 17, \quad 13^4 = 17^2 = 289 \bmod 19 = 289 - 15 \times 19 = 4, \quad 13^6 = 4 \times 17 = 68 \bmod 19 = 11.$$

$$c2 = 7 \times 11 \bmod 19 = 77 \bmod 19 = 77 - 4 \times 19 = 1.$$

Ciphertext (7, 1) — completely different from (8, 8). Same plaintext, different ciphertext.

This is ElGamal's probabilistic nature in action.

Q11 — ElGamal — Second Calculation (p=11, g=2, x=4, m=5, k=3)

(a) Compute h, then (c1, c2).

(b) Decrypt and verify m=5.

(c) If an attacker learns k=3, can they recover m? Explain.

ANSWER — Q11

(a)

$$h = 2^4 \bmod 11 = 16 \bmod 11 = 5$$

$$c1 = 2^x \bmod 11 = 8$$

$$h^k = 5^3 = 125 \bmod 11 = 125 - 11 \times 11 = 4$$

$$c2 = 5 \times 4 \bmod 11 = 20 \bmod 11 = 9$$

Ciphertext: (8, 9)

(b)

$$s = c1^x \bmod p = 8^4 \bmod 11.$$

$$8^2=64 \bmod 11=9. \quad 8^4=9^2=81 \bmod 11=4. \quad s=4.$$

$$s^{-1}: 4 \times y \equiv 1 \pmod{11}. \quad 4 \times 3 = 12 \equiv 1 \rightarrow s^{-1}=3.$$

$$m = 9 \times 3 \bmod 11 = 27 \bmod 11 = 5 \quad \checkmark$$

(c) Yes, if k is leaked the attacker can recover m:

$c1 = g^x \bmod p$ is public and k is now known.

The attacker computes $h^k = (g^x)^k = (g^k)^x$.

Actually they compute h^k directly: $h=5, k=3 \rightarrow h^k=5^3=125 \bmod 11=4$.

Then $m = c2 \times (h^k)^{-1} \bmod p = 9 \times 4^{-1} \bmod 11$.

$$4^{-1} \bmod 11: 4 \times 3 = 12 \equiv 1 \rightarrow 4^{-1}=3. \quad m=9 \times 3 \bmod 11=5 \quad \checkmark$$

This is why k must be secret and never reused. If k is reused for two messages, an attacker gets $c2_1/c2_2 = m1/m2$, leaking the ratio.

Q12 — ElGamal — Reasoning Questions

(a) Why does ElGamal need TWO ciphertext values (c_1 , c_2) while RSA needs only one (c)?

(b) In the decryption step, WHY does computing $s = c_1^x$ remove the blinding from c_2 ?

(c) What happens if an attacker knows k ? What if they know x ?

ANSWER — Q12

(a) Why two values?

The encryptor blinds the message: $c_2 = m \times h^k \bmod p$. To decrypt, the receiver must remove the blinding factor h^k . To reconstruct h^k , the decryptor needs to know k — but k is secret. So the encryptor sends $c_1 = g^k$, allowing the decryptor to compute $(g^k)^x = g^{(kx)} = h^k$. Without c_1 , decryption is impossible. RSA has no such blinding — it directly applies the inverse function $m = c^d$.

(b) Why does $s = c_1^x$ remove the blinding?

$$s = c_1^x = (g^k)^x = g^{(kx)}.$$

$$h^k = (g^x)^k = g^{(xk)} = g^{(kx)} = s.$$

So $s = h^k$ exactly. Multiplying c_2 by s^{-1} :

$$c_2 \times s^{-1} = (m \times h^k) \times (h^k)^{-1} = m \times 1 = m.$$

The blinding factor cancels perfectly.

(c) If attacker knows k : they compute h^k directly and recover $m = c_2 \times (h^k)^{-1}$. Complete break of this specific ciphertext.

If attacker knows x (the secret key): they compute $s = c_1^x$ and decrypt ANY ciphertext ever encrypted with this public key. Complete and permanent break of the entire scheme.

Exercise 5 — Brute Force Key Space

Q13 — Brute Force — 64-bit Key at 2^{32} keys/second

- (a) How many total keys exist for a 64-bit key?
- (b) Worst-case time to try all keys (in seconds and years)?
- (c) Is this estimate conservative or optimistic? Why?
- (d) How does this compare to AES-128? Factor of difficulty?

ANSWER — Q13

(a) 2^{64} total keys $\approx 1.8 \times 10^{19}$.

(b) Time = $2^{64} / 2^{32} = 2^{32}$ seconds.

$2^{32} = 4,294,967,296$ seconds.

$\div (365 \times 24 \times 3600) = \div 31,536,000 \approx 136$ years.

Not immediately infeasible with massive parallel hardware!

(c) Optimistic (too fast in reality) because:

- Assumes constant maximum speed — real hardware has memory bottlenecks, heat limits.
- Does not account for overhead of generating, storing, and checking each key.
- Assumes perfect parallelisation.

In practice, actual time would be longer. But with nation-state resources, 64-bit keys are considered insecure today.

(d) AES-128 vs 64-bit:

Ratio = $2^{128} / 2^{64} = 2^{64} \approx 1.8 \times 10^{19}$ times harder.

At the same 2^{32} keys/s: AES-128 would take $2^{128}/2^{32} = 2^{96}$ seconds $\approx 2.5 \times 10^{21}$ years — completely infeasible.

Q14 — Brute Force — Comparing 64-bit and 128-bit Keys at 2^{50} keys/s

- (a) Time to brute-force a 64-bit key at 2^{50} keys/s (in hours)?
- (b) Time for a 128-bit key at the same speed (in years)?
- (c) How many times harder is 128-bit than 64-bit?
- (d) Does a longer key automatically mean a secure system?

ANSWER — Q14

(a) 64-bit at 2^{50} keys/s:

Time = $2^{64} / 2^{50} = 2^{14}$ seconds = 16,384 seconds $\approx$ **4.5 hours**.

A 64-bit key is trivially broken by a powerful adversary!

(b) 128-bit at 2^{50} keys/s:

Time = $2^{128} / 2^{50} = 2^{78}$ seconds.

$2^{78} / (3.15 \times 10^7 \text{ s/year}) \approx \mathbf{9.5 \times 10^{15} \text{ years}}$.

About 690,000 times the current age of the universe.

(c) Factor: $2^{128} / 2^{64} = 2^{64} \approx \mathbf{1.8 \times 10^{19} \text{ times harder}}$.

Each additional bit DOUBLES the difficulty. Going from 64 to 128 bits adds 64 doublings.

(d) No. Key length is necessary but not sufficient. A system can fail due to:

- Implementation bugs (buffer overflow, timing side-channel)
- IV/nonce reuse (same nonce with AES-GCM = catastrophic)
- Padding oracle attacks (exploiting error messages)
- Weak key generation (non-random seed)

"AES-128 with nonce reuse" is far less secure than "AES-128 used correctly."

Exercise 6 — TLS Cipher Suite Identification

Q15 — Decode TLS 1.2 Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

- (a) What is ECDHE and what does "ephemeral" mean?
- (b) What does RSA do here — is it encrypting data?
- (c) What is AES_256_GCM and what does GCM add?
- (d) What is SHA384 used for?
- (e) This suite provides forward secrecy. Explain what that means.

ANSWER — Q15

(a) ECDHE = Elliptic Curve Diffie-Hellman Ephemeral.

Role: key exchange — used ONCE per session to establish a shared session key.

"Ephemeral" means a brand-new key pair is generated for EVERY connection. The private key is discarded after the handshake. This is what gives forward secrecy.

(b) RSA here = authentication, NOT encryption.

The server uses its RSA private key to SIGN part of the handshake, proving it owns the certificate. The client verifies this signature using the certificate's public key. No data is encrypted with RSA — ECDHE establishes the session key.

(c) AES_256_GCM:

AES-256 = symmetric cipher encrypting all session data after the handshake.

GCM (Galois/Counter Mode) = AEAD mode: provides BOTH confidentiality (encryption) AND integrity (MAC) in a single operation. An attacker cannot modify ciphertext undetected.

(d) **SHA384:** cryptographic hash function used in the TLS PRF (pseudorandom function) for key derivation and in the HMAC for integrity verification during the handshake.

(e) Forward secrecy:

If the server's long-term RSA private key is compromised in the future, an attacker who recorded past sessions CANNOT decrypt them. Why? Because the session key was derived from ephemeral ECDHE values that were discarded immediately — they no longer exist. Each session's security is independent.

Q16 — TLS 1.3 vs TLS 1.2 — Differences and Deprecations

TLS 1.3: **TLS_AES_256_GCM_SHA384**

TLS 1.2: **TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384**

- (a) What is missing from the TLS 1.3 suite name compared to TLS 1.2?
- (b) Why did TLS 1.3 remove those fields?
- (c) Which TLS versions should be disabled and why?
- (d) What does an SSL Labs A+ rating indicate?

ANSWER — Q16

(a) The TLS 1.3 suite is missing:

- The key exchange algorithm (ECDHE)
- The authentication algorithm (RSA)

TLS 1.3 only lists the symmetric cipher (AES_256_GCM) and hash (SHA384).

(b) In TLS 1.3, forward secrecy is MANDATORY — only ephemeral key exchange is allowed. There is no point negotiating the key exchange method because only one category is permitted. Authentication is also handled separately in the certificate.

TLS 1.3 is simpler, faster (1-RTT handshake vs 2-RTT in TLS 1.2), and removes legacy cipher suite negotiation that introduced downgrade attack risks.

(c) Disable TLS 1.0 and TLS 1.1:

- TLS 1.0: vulnerable to BEAST attack, uses MD5/SHA-1 in the PRF.
- TLS 1.1: improved on 1.0 but still uses weak PRF and older cipher suites.
- TLS 1.2: still acceptable with strong cipher suites (no RC4, no CBC without AEAD).
- TLS 1.3: preferred — modern, secure, faster.

(d) SSL Labs A+ indicates:

- Only TLS 1.2 and/or 1.3 supported (1.0 and 1.1 disabled)
- Strong cipher suites only (AEAD, forward secrecy)
- Valid, strong certificate (RSA-2048+ or ECDSA)
- HSTS (HTTP Strict Transport Security) enabled
- No weak keys, no known vulnerabilities
- GitHub, for example, uses TLS 1.3 + RSA-4096 certificate and scores A+.

LAB 10 · MPC, BOOLEAN SECRET SHARING & BEAVER TRIPLES

Exercise 1 — Boolean Secret Sharing

Q17 — Valid Share Pairs and Reconstruction

(a) Secret $a=0$. Give two different valid pairs (a_1, a_2) such that $a_1 \text{ XOR } a_2 = 0$.

(b) Secret $a=1$. Give two different valid pairs.

(c) Given shares $(a_1=1, a_2=0)$: reconstruct a .

(d) Given shares $(a_1=1, a_2=1)$: reconstruct a .

(e) Secret $a=1$, share $a_1=0$ given to Alice. What is Bob's share a_2 ?

ANSWER — Q17

(a) Two pairs for $a=0$:

$(0, 0)$: $0 \text{ XOR } 0 = 0$ ✓

$(1, 1)$: $1 \text{ XOR } 1 = 0$ ✓

(b) Two pairs for $a=1$:

$(0, 1)$: $0 \text{ XOR } 1 = 1$ ✓

$(1, 0)$: $1 \text{ XOR } 0 = 1$ ✓

(c) $a = 1 \text{ XOR } 0 = 1$

(d) $a = 1 \text{ XOR } 1 = 0$

(e) $a_2 = a \text{ XOR } a_1 = 1 \text{ XOR } 0 = 1$.

Verify: $0 \text{ XOR } 1 = 1 = a$ ✓

Why: If $a = a_1 \text{ XOR } a_2$, then $a_2 = a \text{ XOR } a_1$ (XOR is its own inverse).

Q18 — Why a Single Share Reveals Nothing

Alice holds share $a_1=1$. She does not know a_2 .

(a) What are the possible values of secret a from Alice's view? Use probabilities.

(b) What cryptographic primitive does this masking resemble?

(c) Bob holds $a_2=0$. They combine shares. What is a ?

ANSWER — Q18

(a) Alice sees $a_1=1$. But $a = a_1 \text{ XOR } a_2$.

· If $a_2=0$ (which Alice doesn't know): $a = 1 \text{ XOR } 0 = 1$

· If $a_2=1$ (which Alice doesn't know): $a = 1 \text{ XOR } 1 = 0$

Since a_2 is uniformly random (chosen at share creation), both are equally likely:

$$\Pr(a=0 \mid a_1=1) = \Pr(a=1 \mid a_1=1) = 1/2$$

Alice learns NOTHING about a from a_1 alone.

(b) One-Time Pad (OTP).

In OTP: ciphertext = plaintext XOR random_key. With a random key, the ciphertext is uniformly random and reveals nothing about the plaintext. Identically here: $a_1 = a \text{ XOR } r$ where r is random. a_1 is uniformly random and reveals nothing about a .

(c) $a = a_1 \text{ XOR } a_2 = 1 \text{ XOR } 0 = 1$.

Yes — combining both shares immediately reconstructs the secret. This is why the two shares must be held by non-colluding parties.

Q19 — Missing Share Calculation

For each case, find the missing share.

(a) Secret $a=1$, share $a_1=0$. Find a_2 .

(b) Secret $a=0$, share $a_1=1$. Find a_2 .

(c) Secret $a=0$, share $a_1=0$. Find a_2 .

(d) Secret $a=1$, share $a_1=1$. Find a_2 .

ANSWER — Q19

Rule: $a_2 = a \text{ XOR } a_1$ (because $a = a_1 \text{ XOR } a_2 \rightarrow a_2 = a \text{ XOR } a_1$)

(a) $a_2 = 1 \text{ XOR } 0 = 1$. Verify: $0 \text{ XOR } 1 = 1 = a$ ✓

(b) $a_2 = 0 \text{ XOR } 1 = 1$. Verify: $1 \text{ XOR } 1 = 0 = a$ ✓

(c) $a_2 = 0 \text{ XOR } 0 = 0$. Verify: $0 \text{ XOR } 0 = 0 = a$ ✓

(d) $a_2 = 1 \text{ XOR } 1 = 0$. Verify: $1 \text{ XOR } 0 = 1 = a$ ✓

Exercise 2 — XOR vs AND Under Boolean Sharing

Q20 — Local XOR Computation (no communication needed)

Secrets $a=1$ and $b=0$ are shared:

a is shared as $(a_1=0, a_2=1)$ — verify: $0 \text{ XOR } 1 = 1 = a$ ✓

b is shared as $(b_1=1, b_2=1)$ — verify: $1 \text{ XOR } 1 = 0 = b$ ✓

(a) Compute shares of $c = a \text{ XOR } b$. What does Server A compute? Server B?

(b) Reconstruct c . Verify against true $a \text{ XOR } b$.

(c) Why is no communication needed between servers?

ANSWER — Q20

(a) Each server XORs their own shares locally:

Server A: $c_A = a_1 \text{ XOR } b_1 = 0 \text{ XOR } 1 = 1$

Server B: $c_B = a_2 \text{ XOR } b_2 = 1 \text{ XOR } 1 = 0$

(b) Reconstruct: $c = c_A \text{ XOR } c_B = 1 \text{ XOR } 0 = 1$.

True $a \text{ XOR } b = 1 \text{ XOR } 0 = 1$ ✓

(c) XOR is a linear operation under XOR sharing.

Proof: $c_A \text{ XOR } c_B = (a_1 \text{ XOR } b_1) \text{ XOR } (a_2 \text{ XOR } b_2)$

$= (a_1 \text{ XOR } a_2) \text{ XOR } (b_1 \text{ XOR } b_2)$ (XOR is associative and commutative)

$= a \text{ XOR } b$ ✓

Each server only needs their own local shares — no cross-party values required. This makes XOR operations essentially free in terms of communication cost.

Q21 — Why AND Cannot Be Computed Locally

Using: $a=(a1=0, a2=1)$, $b=(b1=1, b2=1)$

(a) Expand $(a1 \text{ XOR } a2) \text{ AND } (b1 \text{ XOR } b2)$ into four AND terms.

(b) Which terms require values from BOTH servers?

(c) Why does this mean AND cannot be computed locally?

(d) What is the solution?

ANSWER — Q21

(a) Expansion (distributive law for bits):

$(a1 \text{ XOR } a2) \text{ AND } (b1 \text{ XOR } b2)$

$= (a1 \text{ AND } b1) \text{ XOR } (a1 \text{ AND } b2) \text{ XOR } (a2 \text{ AND } b1) \text{ XOR } (a2 \text{ AND } b2)$

(b) Cross-party terms:

· $(a1 \text{ AND } b2)$: $a1=0$ held by Server A, $b2=1$ held by Server B $\rightarrow$ CROSS

· $(a2 \text{ AND } b1)$: $a2=1$ held by Server B, $b1=1$ held by Server A $\rightarrow$ CROSS

· $(a1 \text{ AND } b1)$: both at Server A $\rightarrow$ local ✓

· $(a2 \text{ AND } b2)$: both at Server B $\rightarrow$ local ✓

(c) AND is nonlinear — it does not distribute over XOR sharing the way XOR does. The cross-party terms require values that one server does not have, so at least one round of communication between the servers is necessary.

(d) Beaver triples. A precomputed tuple (i, j, k) with $k=i \text{ AND } j$ allows the AND to be computed with just two public openings (of masked values d and e), which reveal nothing about a or b .

Exercise 3 — AND Gate Using a Beaver Triple

Q22 — Why Revealing d and e Is Safe

A Beaver triple (i, j, k) with $k=i \text{ AND } j$ is used. During AND computation:

$d = a \text{ XOR } i$ and $e = b \text{ XOR } j$ are opened (made public).

(a) If $i=1$ and $a=0$, what is d ? Does d reveal a ?

(b) If $i=0$ and $a=0$, what is d ? Does d reveal a ?

(c) Explain in general why d never reveals a .

(d) Which Lecture 9 primitive uses the identical masking idea?

ANSWER — Q22

(a) $d = a \text{ XOR } i = 0 \text{ XOR } 1 = 1$.

An observer sees $d=1$. Do they know a ? Only if they know i .

Since i is secret: $d=1$ is consistent with $a=0, i=1$ OR $a=1, i=0$. Equal probability.

$d=1$ reveals nothing about a .

(b) $d = 0 \text{ XOR } 0 = 0$.

Observer sees $d=0$. Consistent with $a=0, i=0$ OR $a=1, i=1$. Equal probability.

$d=0$ reveals nothing about a .

(c) General argument:

$d = a \text{ XOR } i$ where i is uniformly random in $\{0,1\}$.

For any fixed value of d , both $a=0$ (with $i=d$) and $a=1$ (with $i=d \text{ XOR } 1$) are equally consistent with what the observer sees.

Formally: $\Pr(a=0 \mid d) = \Pr(a=1 \mid d) = 1/2$.

d is statistically independent of a — it leaks zero information.

(d) The One-Time Pad.

OTP: ciphertext = plaintext XOR random_key. Identically here: $d = a \text{ XOR } \text{random}_i$. Both rely on XOR with a uniformly random value. Both give perfect secrecy for the masked value. The critical shared requirement: the mask must be used **only once**.

Q23 — Prove the AND Reconstruction Formula

Starting from $a = d \text{ XOR } i$ and $b = e \text{ XOR } j$, prove:

$$a \text{ AND } b = k \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (d \text{ AND } e)$$

Show every expansion step.

ANSWER — Q23

Step 1: Substitute $a = d \text{ XOR } i$ and $b = e \text{ XOR } j$:

$$a \text{ AND } b = (d \text{ XOR } i) \text{ AND } (e \text{ XOR } j)$$

Step 2: Expand using the distributive law $(X \text{ XOR } Y) \text{ AND } Z = (X \text{ AND } Z) \text{ XOR } (Y \text{ AND } Z)$:

$$= (d \text{ AND } (e \text{ XOR } j)) \text{ XOR } (i \text{ AND } (e \text{ XOR } j))$$

$$= (d \text{ AND } e) \text{ XOR } (d \text{ AND } j) \text{ XOR } (i \text{ AND } e) \text{ XOR } (i \text{ AND } j)$$

Step 3: Substitute $k = i \text{ AND } j$:

$$= (d \text{ AND } e) \text{ XOR } (d \text{ AND } j) \text{ XOR } (i \text{ AND } e) \text{ XOR } k$$

Step 4: Reorder (XOR is commutative):

$$= k \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (d \text{ AND } e) \text{ QED } \checkmark$$

Explanation of each term:

$k = i \text{ AND } j$: the precomputed Beaver triple term — "the AND of the masks."

$d \text{ AND } j$: "masked a" AND "mask for b" — a cross-term made safe because j is revealed.

$e \text{ AND } i$: "masked b" AND "mask for a" — symmetric cross-term.

$d \text{ AND } e$: "masked a" AND "masked b" — computable locally since both d and e are public.

Together these four terms exactly reconstruct $a \text{ AND } b$.

Q24 — AND Formula Drill #1 — $a=0$, $b=1$, triple ($i=1$, $j=0$, $k=0$)

Verify the triple: $i \text{ AND } j = 1 \text{ AND } 0 = 0 = k$ ✓

(a) Compute $d = a \text{ XOR } i$.

(b) Compute $e = b \text{ XOR } j$.

(c) Evaluate: $k \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (d \text{ AND } e)$. Show each term.

(d) Verify: $\text{result} = a \text{ AND } b = 0 \text{ AND } 1 = 0$.

ANSWER — Q24

(a) $d = 0 \text{ XOR } 1 = 1$

(b) $e = 1 \text{ XOR } 0 = 1$

(c) Evaluate each term:

$k = 0$

$d \text{ AND } j = 1 \text{ AND } 0 = 0$

$e \text{ AND } i = 1 \text{ AND } 1 = 1$

$d \text{ AND } e = 1 \text{ AND } 1 = 1$

$\text{Result} = 0 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 1 = 0$

(d) $a \text{ AND } b = 0 \text{ AND } 1 = 0 = \text{result}$ ✓

Q25 — AND Formula Drill #2 — $a=1$, $b=1$, triple ($i=0$, $j=1$, $k=0$)

Verify: $i \text{ AND } j = 0 \text{ AND } 1 = 0 = k$ ✓

(a) Compute d and e .

(b) Evaluate the full formula. Show each term.

(c) Verify $\text{result} = 1$.

ANSWER — Q25

(a) $d = 1 \text{ XOR } 0 = 1$ | $e = 1 \text{ XOR } 1 = 0$

(b) Terms:

$k = 0$

$d \text{ AND } j = 1 \text{ AND } 1 = 1$

$e \text{ AND } i = 0 \text{ AND } 0 = 0$

$d \text{ AND } e = 1 \text{ AND } 0 = 0$

$\text{Result} = 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 0 = 1$

(c) $a \text{ AND } b = 1 \text{ AND } 1 = 1 = \text{result}$ ✓

Q26 — AND Formula Drill #3 — $a=0$, $b=0$, triple ($i=1$, $j=1$, $k=1$)

Verify: $1 \text{ AND } 1 = 1 = k$ ✓

(a) Compute d , e .

(b) Evaluate the formula.

(c) Verify result = 0.

(d) ONLY Bob adds ($d \text{ AND } e$) to his k share. Why is the result still correct?

ANSWER — Q26

(a) $d = 0 \text{ XOR } 1 = 1$ | $e = 0 \text{ XOR } 1 = 1$

(b) Terms:

$k = 1$

$d \text{ AND } j = 1 \text{ AND } 1 = 1$

$e \text{ AND } i = 1 \text{ AND } 1 = 1$

$d \text{ AND } e = 1 \text{ AND } 1 = 1$

Result = $1 \text{ XOR } 1 \text{ XOR } 1 \text{ XOR } 1 = 0$

(c) $a \text{ AND } b = 0 \text{ AND } 0 = 0 = \text{result}$ ✓

(d) The ($d \text{ AND } e$) term is publicly known to both parties (d and e are both revealed). One party adding it to their local share is equivalent to adding it to the shared result — because the XOR reconstruction still holds.

Formally: if Alice holds k_A and Bob holds k_B with $k_A \text{ XOR } k_B = k$, and Bob adds ($d \text{ AND } e$):

New reconstruction: $k_A \text{ XOR } (k_B \text{ XOR } (d \text{ AND } e)) = k \text{ XOR } (d \text{ AND } e)$

This still equals the formula output since ($d \text{ AND } e$) was part of the formula.

Exercise 4 — The Matching Problem

Q27 — Mutual Interest as a Boolean Function

Alice has private bit a (1=interested, 0=not). Bob has private bit b .

(a) What Boolean function represents mutual interest?

(b) Fill in the full truth table.

(c) Why would simply asking each other break privacy?

ANSWER — Q27

(a) $f(a, b) = a \text{ AND } b$

Both must be 1 for a match. If either is 0, the result is 0.

(b) Truth table:

$a=0, b=0 \rightarrow 0$ (neither interested)

$a=0, b=1 \rightarrow 0$ (only Bob interested)

$a=1, b=0 \rightarrow 0$ (only Alice interested)

$a=1, b=1 \rightarrow 1$ (MATCH — both interested)

(c) Privacy problem with direct disclosure:

If Alice says $a=0$ first, Bob learns Alice is not interested — embarrassing/private.

If Bob says $b=1$ first, he reveals interest to someone who may not reciprocate.

The MPC approach ensures: if result=0, neither party learns the OTHER's preference (beyond what the output logically implies). Specifically: if $a=0$ and $b=1$, the result 0 tells Bob the answer is no — but Bob cannot determine WHETHER Alice said no or he said no (in a symmetric version). In the basic two-bit case, if Bob knows $b=1$ and result=0, he can infer $a=0$. But this is "allowed leakage" — it follows from the function output, not from the protocol.

Q28 — Compute Mutual Interest with MPC (Full Walkthrough)

Alice has $a=1$, Bob has $b=1$.

Beaver triple available: $i=0, j=0, k=0$ (verify: $0 \text{ AND } 0 = 0 = k$ ✓)

(a) Alice shares a : she chooses $r=1$. Compute a_1, a_2 .

(b) Bob shares b : he chooses $s=0$. Compute b_1, b_2 .

(c) Compute $d = a \text{ XOR } i$ and $e = b \text{ XOR } j$.

(d) Are d and e safe to reveal? Why?

(e) Evaluate $a \text{ AND } b$ using the Beaver formula.

(f) What does the result tell Alice and Bob?

ANSWER — Q28

(a) Alice shares $a=1$ with $r=1$:

$a_1 = a \text{ XOR } r = 1 \text{ XOR } 1 = 0$ (sent to Server A)

$a_2 = r = 1$ (sent to Server B)

Verify: $0 \text{ XOR } 1 = 1 = a$ ✓

(b) Bob shares $b=1$ with $s=0$:

$b_1 = b \text{ XOR } s = 1 \text{ XOR } 0 = 1$ (sent to Server A)

$b_2 = s = 0$ (sent to Server B)

Verify: $1 \text{ XOR } 0 = 1 = b$ ✓

(c) $d = a \text{ XOR } i = 1 \text{ XOR } 0 = 1$

$e = b \text{ XOR } j = 1 \text{ XOR } 0 = 1$

(d) Yes, safe. $i=0$ and $j=0$ are random masks (from the Beaver triple). $d = a \text{ XOR } i = 1 \text{ XOR } 0 = 1$. An observer sees $d=1$ but without knowing $i=0$, they cannot tell if $a=0$ (with $i=1$) or $a=1$ (with $i=0$) — equal probability. Same logic for e .

(e) Evaluate:

$k = 0$

$d \text{ AND } j = 1 \text{ AND } 0 = 0$

$e \text{ AND } i = 1 \text{ AND } 0 = 0$

$d \text{ AND } e = 1 \text{ AND } 1 = 1$

$a \text{ AND } b = 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 1 = 1$

(f) Result = 1 means **mutual match.** Both Alice and Bob are interested.

The protocol reveals **ONLY** the result — the individual inputs $a=1$ and $b=1$ were never directly shared between the parties.

Q29 — Output Leakage Analysis

- (a) Result=0, Bob knows $b=1$. What can Bob infer about Alice's a ?
- (b) Result=1. What does Bob know about Alice?
- (c) Result=0, Bob knows $b=0$. What does Bob know about Alice?
- (d) What does "no leakage beyond the function output" mean?

ANSWER — Q29

(a) Result=0, $b=1$:

$a \text{ AND } b = 0 \text{ AND } b=1 \rightarrow a \text{ must be } 0$.

Bob can correctly infer **$a=0$** .

Is this a privacy violation? **No** — this information follows LOGICALLY from the function output itself ($a \text{ AND } 1 = 0$ iff $a = 0$). MPC cannot prevent leakage that the function output inherently reveals.

(b) Result=1, $b=1$:

$a \text{ AND } b = 1 \rightarrow$ both must be 1. Bob infers **$a=1$** .

Again, allowed leakage — logically implied by the output.

(c) Result=0, $b=0$:

$a \text{ AND } 0 = 0$ regardless of a . Bob learns NOTHING about Alice's a .

a could be 0 or 1 — the result 0 is consistent with both.

(d) "No leakage beyond the function output":

The MPC protocol guarantees that the ONLY information parties learn is what can be inferred from the function output $f(a,b)$ and their own input. Nothing else.

The protocol itself (the shares, the Beaver triple openings, d and e) does not reveal any additional information about the other party's input beyond what $f(a,b)$ already tells you.

Example: the fact that $d=1$ and $e=1$ in the computation above does NOT reveal that $a=1$ and $b=1$ — it is equally consistent with other (a,b,i,j) combinations.

Exercise 5 — Security Intuition

Q30 — What Ensures Security When d and e Are Revealed

- (a) What property of i and j makes opening d and e safe?
- (b) A Beaver triple (i,j,k) is reused for two computations. What breaks?
- (c) Compare the masking of a by i in MPC with OTP. What shared rule applies?

ANSWER — Q30

(a) i and j are uniformly random and SECRET.

They were generated in the offline phase before real inputs arrived. Each serves as a one-time random mask for one AND computation. Since i is uniform random, $d = a \text{ XOR } i$ is also uniform random — statistically independent of a. Zero information about a leaks through d.

(b) Reusing a triple breaks security:

First use: $d1 = a1 \text{ XOR } i$, $e1 = b1 \text{ XOR } j$ (revealed publicly)

Second use (same i,j!): $d2 = a2 \text{ XOR } i$, $e2 = b2 \text{ XOR } j$

Attacker computes: $d1 \text{ XOR } d2 = (a1 \text{ XOR } i) \text{ XOR } (a2 \text{ XOR } i) = a1 \text{ XOR } a2$.

The attacker now knows the XOR of both private inputs — a serious leakage.

Similarly for $e1 \text{ XOR } e2 = b1 \text{ XOR } b2$.

Rule: each Beaver triple must be used exactly once.

(c) Comparison with OTP:

OTP: ciphertext = plaintext XOR key. Rule: never reuse the key.

Beaver masking: $d = a \text{ XOR } i$. Rule: never reuse i.

Both provide perfect information-theoretic secrecy for the masked value.

Both fail catastrophically under key/mask reuse.

Shared rule: the mask must be uniformly random and used exactly once.

Q31 — Offline vs Online Phase — Why Separate Them?

(a) What is computed in the offline phase?

(b) What is computed in the online phase?

(c) Why is the separation useful?

(d) What additional protection is needed against a malicious (not just curious) server?

ANSWER — Q31

(a) Offline (preprocessing) phase:

Generate random bits i and j , compute $k = i \text{ AND } j$.

Secret-share all three values: i , j , k are each split between the two servers.

This is done BEFORE the real inputs (a and b) are known.

It is the expensive part — requires interaction or trusted hardware.

(b) Online phase:

Receive actual inputs a and b (as secret shares).

Compute $d = a \text{ XOR } i$ and $e = b \text{ XOR } j$.

Open d and e to both parties (two broadcasts).

Each party applies the AND formula locally to get their share of the output.

Very fast — only two broadcasts and local computation.

(c) Why separate?

The expensive work (triple generation) is done once ahead of time, possibly during idle periods. When real data arrives, the online phase only needs two broadcast rounds — extremely fast. This is critical for latency-sensitive MPC applications.

(d) Against malicious servers:

An honest-but-curious server follows the protocol but tries to learn extra info.

A malicious server may send WRONG shares or a bad triple to corrupt the result.

Protection requires: authenticated shares (MACs on each share), consistency checks (verify the triple satisfies $k=i \text{ AND } j$), or zero-knowledge proofs (prove correct behaviour without revealing secrets).

Q32 — Two-Server Model — Trust, Collusion and Applications

- (a) Why can one corrupted server not learn the secret?
- (b) What happens if both servers collude?
- (c) How does two-server MPC differ from a trusted third party?
- (d) Give a real example where MPC is the right tool.

ANSWER — Q32

(a) One corrupted server:

Each server holds only ONE share (e.g. $a_A = a \text{ XOR } r$). The share alone is uniformly random — equally consistent with any secret value. Without the other share, the corrupted server gains zero information about a .

(b) Both servers collude:

They combine their shares: $a_A \text{ XOR } a_B = (a \text{ XOR } r) \text{ XOR } r = a$.

The secret is immediately reconstructed. **Security completely fails.**

This is why non-collusion must be enforced architecturally — e.g. servers in different jurisdictions, different organisations, different legal systems.

(c) Two-server MPC vs Trusted Third Party:

Trusted Third Party (TTP): all parties send raw inputs to one trusted central server. The TTP computes $f(a,b)$ and sends back the result. Simple — but the TTP sees everything.

Two-server MPC: NO single entity ever sees all raw inputs. Each server sees only a share. Privacy is maintained as long as servers do not collude.

Tradeoff: MPC is more complex and slower; TTP is simpler but requires absolute trust.

(d) Real example: Medical research.

Two hospitals want to jointly train a machine learning model on patient data. Legal and ethical rules prohibit sharing raw patient records. With MPC: each hospital holds shares of its own data. The two-server protocol computes aggregate statistics or model updates without either hospital seeing the other's patient records.

PART 1 (CONTINUED) — REFERENCE CARD ADDITIONS

5 · VIGENÈRE CIPHER (Lecture 9 Slides)

What it is: The Vigenère cipher is a **polyalphabetic substitution cipher**. Instead of one fixed shift (Caesar), it uses a repeating keyword where each letter of the key determines the shift for that position. This means the same plaintext letter can produce different ciphertext letters depending on its position — making simple frequency analysis harder than against Caesar.

Why it matters: The slides present this as the step between Caesar and OTP. The key insight is: **repeating keys leak structure**. If you can figure out the key length, you reduce the problem to several Caesar ciphers. The one-time pad fixes this by making the key as long as the message and never repeating it.

How to encrypt — step by step:

1	Write plaintext	e.g. CRYPTOGRAPHY IS FUN
2	Repeat key under it	Key = CAFE → repeat: CAFECAFECAFE CA FEC
3	For each position	Add the letter positions mod 26. A=0, B=1, ..., Z=25.
4	C(2) + C(2) = 4 = E	Plaintext C(2) + Key C(2) = 4 → E
5	R(17) + A(0) = 17 = R	Key A means shift 0 – plaintext letter unchanged
6	Y(24) + F(5) = 29 mod 26 = 3 = D	Wrap around: 29 mod 26 = 3 = D
7	P(15) + E(4) = 19 = T	Result so far: E R D T ...

Encryption formula:

Encrypt	$C = (P + K) \bmod 26$ (P = plaintext letter pos, K = key letter pos)
Decrypt	$P = (C - K + 26) \bmod 26$ (+26 ensures no negative)
Key repeats	Key position = message_position mod key_length
Slide example	CRYPTOGRAPHY IS FUN + CAFECAFECAFE CA FEC = ERDTVOKVCPMC KS KUP

Letter-to-number reference (A=0 ... Z=25):

A=0	B=1	C=2	D=3	E=4	F=5	G=6	H=7	I=8	J=9	K=10	L=11	M=12
N=13	O=14	P=15	Q=16	R=17	S=18	T=19	U=20	V=21	W=22	X=23	Y=24	Z=25

Worked example from slide — CRYPTOGRAPHY with key CAFE:

Pos	Plain	P#	Key	K#	P+K	mod26	Cipher
1	C	2	C	2	4	4	E
2	R	17	A	0	17	17	R
3	Y	24	F	5	29	3	D
4	P	15	E	4	19	19	T
5	T	19	C	2	21	21	V
6	O	14	A	0	14	14	O
7	G	6	F	5	11	11	K
8	R	17	E	4	21	21	V
9	A	0	C	2	2	2	C
10	P	15	A	0	15	15	P
11	H	7	F	5	12	12	M
12	Y	24	E	4	28	2	C

NOTE: Vigenère decrypt: reverse the process — subtract the key letter position mod 26. If result is negative add 26. Caesar is Vigenère with a single-letter key. OTP is Vigenère with a key as long as the message that is never reused — perfect secrecy.

6 · ONE-TIME PAD & XOR (Lecture 9 Slides)

What it is: The One-Time Pad (OTP) is the only encryption scheme with **information-theoretic (perfect) secrecy**. Each plaintext bit is XORed with a uniformly random key bit. With a proper OTP, an adversary with unlimited computing power cannot determine the plaintext because every possible plaintext of that length is equally consistent with the observed ciphertext.

Why it matters: The slides use OTP as the bridge to MPC — the same random masking idea (a XOR random_key) appears directly in Boolean secret sharing. The practical problem is key management: the key must be as long as the message, truly random, shared securely, and **never reused**.

Encrypt	$C = P \text{ XOR } K$ (bit by bit)
Decrypt	$P = C \text{ XOR } K$ (XOR is its own inverse)
XOR rule	$0 \text{ XOR } 0 = 0 \mid 0 \text{ XOR } 1 = 1 \mid 1 \text{ XOR } 0 = 1 \mid 1 \text{ XOR } 1 = 0$
Key cancel	$(P \text{ XOR } K) \text{ XOR } K = P \text{ XOR } (K \text{ XOR } K) = P \text{ XOR } 0 = P$
Reuse leak	$C1 \text{ XOR } C2 = (P1 \text{ XOR } K) \text{ XOR } (P2 \text{ XOR } K) = P1 \text{ XOR } P2$ — KEY CANCELS OUT
Slide example	Plaintext: 1001011 Key: 1100010 Ciphertext: 0101001

Four OTP requirements (all must hold for perfect secrecy):

1	Random key	Key bits must be uniformly random. Predictable key = statistical attack.
2	Key $\geq$ message	Every plaintext bit needs a fresh key bit. Shorter key that repeats = Vigenère-style weakness.
3	Never reuse	Reusing key for two messages: $C1 \text{ XOR } C2 = P1 \text{ XOR } P2$ — key cancels, exposes plaintext relationship.
4	Secure sharing	Both parties must already share the key securely. OTP moves the problem to key distribution.

XOR truth table (memorise this — tested directly in MCQs):

A	B	A XOR B	meaning
0	0	0	same bits $\rightarrow 0$
0	1	1	different $\rightarrow 1$
1	0	1	different $\rightarrow 1$
1	1	0	same bits $\rightarrow 0$

Slide example worked bit by bit:

Plaintext	1 0 0 1 0 1 1
Key	1 1 0 0 0 1 0
XOR = CT	0 1 0 1 0 0 1
Decrypt CT XOR K	0 1 0 1 0 0 1 XOR 1 1 0 0 0 1 0 = 1 0 0 1 0 1 1 ✓

NOTE: OTP vs AES: OTP = perfect secrecy but impractical (key as long as message, one use only). AES = computationally secure, practical, reusable key. OTP vs Vigenère: Vigenère repeats the key (breakable); OTP never repeats (unbreakable).

7 · CRYPTOGRAPHIC GOALS & COMMON PITFALLS (Lecture 9 Notes)

What cryptography provides and what it does NOT automatically provide: This is one of the most tested conceptual areas. The exam repeatedly asks you to identify which security property a mechanism provides and to spot incorrect claims.

Security properties and the mechanisms that provide them:

Confidentiality	Unauthorised parties cannot read the content. Provided by: encryption.
Integrity	Unauthorised changes are detectable. Provided by: authenticated encryption (AEAD), MAC, or digital signature. Raw encryption does NOT automatically provide this.
Authenticity	Receiver can verify origin. Provided by: MAC (shared secret) or digital signature (public key).
Non-repudiation	Signer cannot deny a valid signature. Provided by: digital signatures + PKI.

Common exam pitfall statements — all **WRONG** (memorise these):

WRONG statement	CORRECT answer
"AES-256 has a 256-bit block size."	AES ALWAYS has a 128-bit block. The number in the name is the KEY size. AES-128=10 rounds, AES-192=12, AES-256=14.

"Encryption guarantees integrity."	Raw encryption only provides confidentiality. To detect tampering you need AEAD (e.g. AES-GCM), a MAC (e.g. CMAC/HMAC), or a digital signature.
"A public key can be trusted because it is public."	Public availability does not prove identity. The key must be authenticated — e.g. via a certificate from a trusted CA.
"OTP is weak because XOR is simple."	OTP provides PERFECT SECRECY when conditions are met. Simplicity of XOR is irrelevant; the security comes from the random key.
"RSA mathematics alone is enough for production."	Textbook RSA is deterministic and malleable. Production needs RSA-OAEP (encryption) or RSA-PSS (signatures).
"A standardised protocol can never be broken."	Standardisation improves review and interoperability but does NOT guarantee security forever. EAXprime was standardised then broken in 2012.
"Symmetric encryption is always weaker than public-key."	AES-128 is considered unbreakable in practice. RSA-1024 is considered BROKEN. Key length and construction matter, not the category.
"ChaCha20 is for historical use only."	ChaCha20-Poly1305 is a MODERN AEAD construction used in TLS 1.3, VPNs, SSH, and mobile networks.

Mechanism comparison — what each provides:

Mechanism	Keys	Confidentiality	Integrity	Authenticity	Non-repudiation
Symmetric encryption	Shared secret	✓	✗	✗	✗
MAC / CMAC / HMAC	Shared secret	✗	✓	✓ (among key-holders)	✗
Digital signature	Private/public pair	✗	✓	✓	✓
Authenticated encryption (AEAD)	Shared secret	✓	✓	✓ (among key-holders)	✗
OTP	Random key = msg length	✓ (perfect)	✗	✗	✗
RSA-OAEP	Public/private pair	✓	✗	✗	✗
RSA-PSS signature	Private/public pair	✗	✓	✓	✓

8 · ADDITIVE SECRET SHARING (Lecture 10 Slides)

What it is: Additive secret sharing splits a numeric secret x into two shares x_1 and x_2 such that $x = x_1 + x_2$. Unlike Boolean sharing (XOR), addition is used. This makes additive sharing natural for arithmetic computations: salary sums, averages, machine learning activations, and other numeric functions.

How it connects: The slides introduce additive sharing as the extension of Boolean sharing to handle "more complicated arithmetic functions." Where Boolean sharing uses XOR (good for bit circuits like AND/OR), additive sharing uses addition mod q (good for numeric arithmetic). Beaver triples work for both — the AND/multiplication gate is the expensive operation in both settings.

Split	Choose random x_1 . Set $x_2 = x - x_1 \pmod{q}$. Give x_1 to Server A, x_2 to Server B.
Reconstruct	$x = x_1 + x_2 \pmod{q}$. Both servers send their share; sum = original.
Add locally	To share $(a + b)$: Server A computes a_1+b_1 , Server B computes a_2+b_2 . No communication.
Multiply	Requires Beaver-style preprocessing (multiplication triple). Not free.
vs Boolean	Boolean: $x = x_1 \text{ XOR } x_2$ (bits, XOR free, AND costs). Additive: $x = x_1+x_2$ (numbers, + free, $\times$ costs).
Slide eq	$x = x_1 + x_2$ used for ML activations (ReLU), averages, argmax, salary comparison

Additive sharing reconstruction drill (from MCQ bank):

x_1	x_2	$x = x_1 + x_2$
7	5	12
10	3	13
0	9	9

14	6	20
21	4	25
8	8	16
31	2	33
12	19	31

Boolean vs Additive — comparison table:

Property	Boolean XOR sharing	Additive sharing
Formula	$a = a_1 \text{ XOR } a_2$	$x = x_1 + x_2 \pmod{q}$
Good for	Bit circuits (AND gates, XOR gates)	Arithmetic (sums, averages, ML)
Free operation	XOR — each party operates locally	Addition — each party operates locally
Expensive operation	AND — needs Beaver triple	Multiplication — needs Beaver-style triple
Reconstruction	XOR the two shares	Add the two shares
Example use	Matching ($a \text{ AND } b$), set intersection	Salary average, score sum, ReLU activation

MPC applications mentioned on slides:

Salary comparison	$x > y$ — comparison circuit
Matching	$a \text{ AND } b$ — Boolean circuit
Set intersection	$X \cap Y$ — PSI protocol
Average score	$(x_1 + x_2 + \dots + x_n) / n$ — additive sharing + division
Winner determination	$\text{argmax}(s_1, \dots, s_n)$ — comparison
ML activation	$\text{ReLU}(x) = \max(0, x)$ — arithmetic comparison
AES / Hash / MAC by MPC	Cryptographic functions evaluated on shared inputs

9 · STANDARDISATION — AES / EAXprime / CMAC (Lecture 9 & 10 Slides)

What it is: Cryptographic standardisation is the process by which algorithms, modes, and protocols are publicly specified, reviewed, and adopted by bodies such as NIST, ANSI, or IETF. Standards enable interoperability, concentrate expert scrutiny, and provide a shared security baseline. The slides end Lecture 9 and Lecture 10 on this topic — and the exam tests it with 18+ questions.

The key lesson: Standardisation improves confidence but is **not a permanent security guarantee**. EAXprime was standardised, deployed, and then broken. Security requires: reviewed standards + correct implementation + ongoing analysis + migration readiness.

AES / FIPS PUB 197	Standardised in 2001/2002 under FIPS PUB 197. Selected through a public competition with 15 submissions, 5 finalists. Rijndael (by Daemen and Rijmen) won. Serpent (Anderson, Biham, Knudsen) was runner-up. No practical attack on full AES is known.
EAXprime (EAX')	Standardised in ANSI C12.22 for electricity-meter data transport. In 2012 Minematsu, Lucks, Morita, and Iwata broke it and warned against use. Lesson: standardised $\neq$ safe forever.
CMAC	Cipher-based MAC standardised in NIST SP 800-38B in 2005. Uses AES to produce an integrity/authenticity tag. Used in low-power and wireless devices: WiFi, Bluetooth. Provides integrity + authenticity, NOT confidentiality.
AES vs EAXprime	AES = success story: 20+ years, no break, still the standard. EAXprime = warning story: standardised, deployed, broken. Both support the lesson: use standards AND keep reviewing them.
Good deployment	Use reviewed standards. Protect keys. Choose correct modes (AEAD not raw ECB). Track updates. Do not invent custom crypto. "AES" alone is incomplete — state the mode (GCM, CCM, etc).

NOTE: Exam trap: "A standardised protocol is always secure." FALSE — EAXprime. "FIPS PUB 197" = AES standard. "NIST SP 800-38B" = CMAC standard. "ANSI C12.22" = the standard that contained EAXprime.

PART 2 (CONTINUED) — ADDITIONAL EXERCISES WITH WORKED ANSWERS

VIGENÈRE CIPHER · Exercises

Exercise — Vigenère Reference Method

Q33 — Vigenère — Encrypt a word (key = DOG)

Using key DOG, encrypt the plaintext: **CAT**

- (a) Write the key repeating under the plaintext.
- (b) Convert each letter pair to numbers, add mod 26, convert back.
- (c) State the ciphertext.

ANSWER — Q33

Key repeat: CAT / DOG (one letter per position)

(a) Letter pairs:

Position 1: $C(2) + D(3) = 5 = F$

Position 2: $A(0) + O(14) = 14 = O$

Position 3: $T(19) + G(6) = 25 = Z$

(c) Ciphertext: **FOZ**

Explanation: Vigenère adds the key letter position to the plaintext letter position mod 26 at each index. Key D means shift by 3, O means shift by 14, G means shift by 6. Unlike Caesar, each position uses a different shift — that is what makes it polyalphabetic.

Q34 — Vigenère — Encrypt with longer key (key = KEY, plaintext = HELLO)

- (a) Repeat key KEY under HELLO.
- (b) Calculate each ciphertext letter step by step.
- (c) State the ciphertext.

ANSWER — Q34

Key repeat: H E L L O / K E Y K E

$H(7) + K(10) = 17 = R$

$E(4) + E(4) = 8 = I$

$L(11) + Y(24) = 35 \bmod 26 = 9 = J$

$L(11) + K(10) = 21 = V$

$O(14) + E(4) = 18 = S$

Ciphertext: **RIJVS**

Explanation: When $P+K \geq 26$, subtract 26 (mod 26). Position 3: $L(11)+Y(24)=35$; $35-26=9=J$. The key repeats from position 1 after its length is exhausted.

Q35 — Vigenère — Encrypt with key = CAFE (slide example)

Encrypt **CRYPTOGRAPHY** using key CAFE.

Show the key repeat and each calculation. This is the exact slide example.

ANSWER — Q35

Key repeat: C R Y P T O G R A P H Y / C A F E C A F E C A F E

$C(2)+C(2)=4=E$ | $R(17)+A(0)=17=R$ | $Y(24)+F(5)=29 \bmod 26=3=D$ | $P(15)+E(4)=19=T$

$T(19)+C(2)=21=V$ | $O(14)+A(0)=14=O$ | $G(6)+F(5)=11=K$ | $R(17)+E(4)=21=V$

$A(0)+C(2)=2=C$ | $P(15)+A(0)=15=P$ | $H(7)+F(5)=12=M$ | $Y(24)+E(4)=28 \bmod 26=2=C$

Ciphertext: ERDTVOKVCPMC (matches the slide exactly)

Why CAFE produces E at position 1: C is the first key letter. $C=2$. Plaintext $C=2$. $2+2=4=E$.

Q36 — Vigenère — Decrypt (key = CAFE, ciphertext = ERDTVOKVCPMC)

Decrypt the ciphertext **ERDTVOKVCPMC** using key CAFE.

(a) State the decryption formula.

(b) Show the first four letter calculations.

(c) What is the plaintext?

ANSWER — Q36

(a) Decryption formula: $P = (C - K + 26) \bmod 26$

Subtract the key letter position; add 26 if the result is negative.

(b) First four positions:

$E(4) - C(2) = 2 = C$

$R(17) - A(0) = 17 = R$

$D(3) - F(5) = -2 + 26 = 24 = Y$

$T(19) - E(4) = 15 = P$

(c) Plaintext: CRYPTOGRAPHY

Explanation: Decryption is the exact reverse of encryption. The +26 before mod 26 ensures we never get a negative number. $D(3) - F(5) = -2$; $-2 + 26 = 24 = Y$.

Q37 — Vigenère — Conceptual: Why is Vigenère stronger than Caesar?

(a) Explain why frequency analysis breaks Caesar but struggles with Vigenère.

(b) Why is Vigenère still breakable despite being polyalphabetic?

(c) How does OTP fix the weakness of Vigenère?

ANSWER — Q37

(a) In Caesar, every plaintext E always maps to the same ciphertext letter. Letter frequencies are preserved — the most frequent ciphertext letter is almost certainly E. In Vigenère, E can map to E, F, G, H... depending on position and key letter. Frequencies are distributed across multiple alphabets, making simple single-letter frequency analysis ineffective.

(b) Vigenère is still breakable because **the key repeats**. An analyst can estimate the key length (using Kasiski test or index of coincidence), then divide the ciphertext into groups — each group was encrypted with the same key letter, reducing each group to a Caesar cipher. Modern computation makes this fast.

(c) OTP uses a key as long as the message that is **never repeated**. There is no pattern to find. Each key bit is used exactly once. Even with unlimited computation, every possible plaintext is equally consistent with the ciphertext — perfect secrecy. The cost: the key distribution problem.

ONE-TIME PAD & XOR CALCULATIONS · Exercises

Exercise — XOR Bit Calculations

Q38 — XOR Encrypt — Slide Example (1001011 XOR 1100010)

This is the exact example from the Lecture 9 slides.

Plaintext: **1001011**

Key: **1100010**

(a) Compute the ciphertext bit by bit.

(b) Verify: decrypt the ciphertext using the same key.

ANSWER — Q38

(a) XOR rule: same bits → 0, different bits → 1

P: 1 0 0 1 0 1 1

K: 1 1 0 0 0 1 0

C: 0 1 0 1 0 0 1

Ciphertext: 0101001

(b) Decrypt: C XOR K

C: 0 1 0 1 0 0 1

K: 1 1 0 0 0 1 0

P: 1 0 0 1 0 1 1 ✓ (original plaintext recovered)

Why XOR decrypts: $(P \text{ XOR } K) \text{ XOR } K = P \text{ XOR } (K \text{ XOR } K) = P \text{ XOR } 0 = P$. XOR is its own inverse — applying it twice with the same key cancels out.

Q39 — XOR Calculations — Six Drills

Compute each XOR result. Show bit-by-bit working.

(a) Plaintext 1010 XOR Key 1100

(b) Plaintext 11110000 XOR Key 01010101

(c) Plaintext 001101 XOR Key 101011

(d) Plaintext 10110101 XOR Key 00111100

(e) Plaintext 111000 XOR Key 101010

(f) Ciphertext 0101001 XOR Key 1100010 = plaintext (decrypt)

ANSWER — Q39

(a) 1010 XOR 1100:

1 XOR 1=0 | 0 XOR 1=1 | 1 XOR 0=1 | 0 XOR 0=0

Result: 0110

(b) 11110000 XOR 01010101:

1XOR0=1 | 1XOR1=0 | 1XOR0=1 | 1XOR1=0 | 0XOR0=0 | 0XOR1=1 | 0XOR0=0 | 0XOR1=1

Result: 10100101

(c) 001101 XOR 101011:

0XOR1=1 | 0XOR0=0 | 1XOR1=0 | 1XOR0=1 | 0XOR1=1 | 1XOR1=0

Result: 100110

(d) 10110101 XOR 00111100:

1XOR0=1 | 0XOR0=0 | 1XOR1=0 | 1XOR1=0 | 0XOR1=1 | 1XOR1=0 | 0XOR0=0 | 1XOR0=1

Result: 10001001

(e) 111000 XOR 101010:

1XOR1=0 | 1XOR0=1 | 1XOR1=0 | 0XOR0=0 | 0XOR1=1 | 0XOR0=0

Result: 010010

(f) 0101001 XOR 1100010 (decrypt):

0XOR1=1 | 1XOR1=0 | 0XOR0=0 | 1XOR0=1 | 0XOR0=0 | 0XOR1=1 | 1XOR0=1

Result: 1001011 (original plaintext — matches slide example ✓)

Q40 — OTP Conditions — What breaks perfect secrecy?

For each scenario, state whether it violates OTP conditions and what goes wrong.

(a) A 2000-bit pad is used to encrypt a 5000-bit message by repeating the pad.

(b) The same pad is used to encrypt two different messages.

(c) The pad is randomly generated and as long as the message, used once.

(d) The pad is a predictable sequence (e.g. 010101... repeating).

ANSWER — Q40

(a) VIOLATION — key shorter than message and reused.

A proper OTP requires key length $\geq$ message length. Repeating a 2000-bit pad over a 5000-bit message is equivalent to Vigenère with a 2000-bit key — the repeated structure is exploitable. Perfect secrecy is lost.

(b) VIOLATION — key reuse.

$C1 = P1 \text{ XOR } K$, $C2 = P2 \text{ XOR } K$.

$C1 \text{ XOR } C2 = P1 \text{ XOR } P2$. The key cancels out completely.

An attacker sees the XOR of both plaintexts — significant leakage.

Known-plaintext attack: if P1 is known, $P2 = (C1 \text{ XOR } C2) \text{ XOR } P1$.

(c) VALID — all OTP conditions satisfied.

Random key ✓, key $\geq$ message length ✓, used only once ✓.

This provides perfect (information-theoretic) secrecy.

(d) VIOLATION — key is not random.

A predictable pattern (010101...) is not uniformly random. The attacker can guess the key structure and recover the plaintext statistically. Perfect secrecy requires every key bit to be independently and uniformly random.

Q41 — OTP vs AES vs Vigenère — Three-Way Comparison

(a) In which situations would you choose OTP over AES?

(b) A student says "OTP is impractical so it must be weak." Correct them.

(c) Describe one real historical use of OTP mentioned in the slides.

ANSWER — Q41

(a) OTP over AES when:

- The communicating parties can securely pre-share a very long key.
- The security requirement is absolute (information-theoretic, not computational).
- The number of messages is small and known in advance.
- Practical example: diplomatic hot-lines, classified government channels.
- In practice, AES is almost always preferred because key management for OTP is operationally infeasible at scale.

(b) Correction:

Practical difficulty and cryptographic security are completely separate concepts. OTP has been mathematically proven to achieve perfect secrecy (Shannon, 1949). An adversary with unlimited computing power CANNOT break a properly used OTP. The difficulty is operational (key length, distribution, storage) — not the encryption itself. AES is easier to deploy but relies on computational assumptions (breaking it would require infeasible brute force). OTP makes no such assumptions.

(c) Moscow-Washington hotline (1960s).

The slides explicitly mention that OTP was used for the Telegram hotline between Moscow and Washington D.C. in the 1960s — a critical diplomatic channel where absolute secrecy was required and key pre-sharing was operationally feasible.

CRYPTOGRAPHIC GOALS & COMMON PITFALLS · Exercises

Exercise — Spot the Error

Q42 — Common Pitfalls — True or False + Correction

For each statement, say TRUE or FALSE and correct it if false.

- (a) "AES-256 provides a 256-bit block size for stronger encryption."
- (b) "Encrypting a message with AES-GCM provides both confidentiality and integrity."
- (c) "A public key can be safely used because it is publicly available."
- (d) "OTP is cryptographically weak because XOR is too simple an operation."
- (e) "Textbook RSA is secure enough for production if 2048-bit keys are used."

ANSWER — Q42

(a) FALSE.

AES ALWAYS uses a 128-bit block — for all three key sizes (128, 192, 256 bit). "AES-256" means a 256-bit KEY with 14 rounds. Block size is fixed at 128 bits. Exam trap: never confuse key size with block size.

(b) TRUE.

AES-GCM is an Authenticated Encryption with Associated Data (AEAD) mode. It provides confidentiality (the AES encryption part) and integrity/authenticity (the GCM Galois authentication tag). If an attacker modifies the ciphertext, the tag verification fails and the message is rejected.

(c) FALSE.

A public key being public tells you nothing about who it belongs to. Without authentication (a certificate from a trusted CA, a fingerprint verification, or another trusted channel), a public key could belong to an attacker. This is the basis of man-in-the-middle attacks against key exchange.

(d) FALSE.

OTP achieves perfect (information-theoretic) secrecy. Shannon proved in 1949 that a correctly used OTP cannot be broken even with unlimited computation. The simplicity of XOR is irrelevant — the security comes from the perfectly random key. "Impractical" ≠ "insecure."

(e) FALSE.

Key length is necessary but not sufficient. Textbook RSA is deterministic (same message always produces same ciphertext) and malleable (ciphertexts can be manipulated without knowing the plaintext). Even with 2048-bit keys, textbook RSA fails IND-CPA security. Production needs RSA-OAEP (encryption) or RSA-PSS (signing).

Q43 — Security Properties — Match Mechanism to Property

For each scenario, identify which cryptographic mechanism is needed and why.

- (a) Alice wants to send a message that only Bob can read.
- (b) Alice wants Bob to be able to verify the message was not modified in transit.
- (c) Alice wants to prove to a judge that she signed a document.
- (d) Alice and Bob want both confidentiality AND integrity over an untrusted channel.
- (e) A WiFi access point needs to verify the integrity of management frames from devices.

ANSWER — Q43

(a) Confidentiality only → Encryption.

Use RSA-OAEP (asymmetric, for key transport) or AES (symmetric, for bulk data). Note: encryption alone does NOT detect modification.

(b) Integrity/authenticity (no confidentiality needed) → MAC or digital signature.

If Alice and Bob share a secret key: use HMAC or CMAC. If the verification must be public (anyone can check): use a digital signature (RSA-PSS or ECDSA).

(c) Non-repudiation → Digital signature.

Only a digital signature provides non-repudiation. A MAC uses a shared key — either party could have produced the tag, so neither can prove the other did it. A digital signature uses a private key only Alice holds — she cannot deny signing.

(d) Confidentiality + integrity together → Authenticated Encryption (AEAD).

Use AES-GCM or ChaCha20-Poly1305. These provide both in a single operation. Never use raw AES (CBC without a MAC) for this — it provides only confidentiality and is vulnerable to padding oracle and modification attacks.

(e) Integrity in a constrained/wireless environment → CMAC.

CMAC (standardised in NIST SP 800-38B) is specifically designed for low-power devices and wireless protocols including WiFi and Bluetooth. It provides integrity and authenticity using AES as the underlying block cipher.

Q44 — Symmetric vs Asymmetric — Classification Drill

Classify each scheme as Symmetric, Asymmetric/Public-Key, or Neither.

Then state the primary purpose (encryption, MAC, signature, key exchange).

(a) AES (b) RSA-OAEP (c) HMAC (d) ElGamal (e) CMAC (f) ChaCha20-Poly1305 (g) ECDHE (h) DES (i) OTP (j) RSA-PSS

ANSWER — Q44

(a) **AES — Symmetric — Encryption** (block cipher, shared key)

(b) **RSA-OAEP — Asymmetric — Encryption** (public key encrypts, private key decrypts)

(c) **HMAC — Symmetric — MAC/Integrity** (shared secret key, produces authentication tag)

(d) **ElGamal — Asymmetric — Encryption** (discrete log based, probabilistic)

(e) **CMAC — Symmetric — MAC/Integrity** (AES-based MAC, used in WiFi/Bluetooth)

(f) **ChaCha20-Poly1305 — Symmetric — AEAD** (stream cipher + authenticator, TLS 1.3)

(g) **ECDHE — Asymmetric — Key Exchange** (not encryption; establishes shared secret)

(h) **DES — Symmetric — Encryption (BROKEN)** (56-bit key, obsolete, do not use)

(i) **OTP — Symmetric — Encryption (perfect secrecy, impractical)**

(j) **RSA-PSS — Asymmetric — Digital Signature** (private key signs, public key verifies)

Key distinctions to remember:

- HMAC/CMAC = integrity + authenticity but NOT non-repudiation (shared key).
- Digital signature = integrity + authenticity + non-repudiation (private key).
- ECDHE = key exchange only — not encryption or signing.
- AEAD = confidentiality + integrity in one (AES-GCM, ChaCha20-Poly1305).

ADDITIVE SECRET SHARING · Exercises

Exercise — Additive Sharing Calculations

Q45 — Additive Sharing — Reconstruct x from shares

For each pair of shares, reconstruct the secret $x = x_1 + x_2$.

(a) $x_1=7$, $x_2=5$ (b) $x_1=0$, $x_2=9$ (c) $x_1=14$, $x_2=6$

(d) $x_1=21$, $x_2=4$ (e) $x_1=8$, $x_2=8$ (f) $x_1=12$, $x_2=19$

ANSWER — Q45

Rule: $x = x_1 + x_2$ (simple addition, no mod for these examples)

(a) $7 + 5 = 12$

(b) $0 + 9 = 9$

(c) $14 + 6 = 20$

(d) $21 + 4 = 25$

(e) $8 + 8 = 16$

(f) $12 + 19 = 31$

Compare with Boolean sharing: Boolean uses $x = x_1 \text{ XOR } x_2$. Additive uses $x = x_1 + x_2$. Different operations, same principle: individual shares hide the secret; combining reveals it.

Q46 — Additive vs Boolean — Choose the Right Sharing

- (a) Alice and Bob want to compute the SUM of their private salaries without revealing individual salaries. Which sharing is more natural?
- (b) Alice and Bob want to compute whether they BOTH like each other (AND). Which sharing is more natural?
- (c) Explain why addition is "free" under additive sharing.
- (d) Why does multiplication under additive sharing still require communication?

ANSWER — Q46

(a) Additive sharing is more natural for sums.

Represent Alice's salary as (a_1, a_2) with $a = a_1 + a_2$.

Represent Bob's salary as (b_1, b_2) with $b = b_1 + b_2$.

Server A computes $a_1 + b_1$ locally. Server B computes $a_2 + b_2$ locally.

Reconstruction: $(a_1+b_1) + (a_2+b_2) = a + b$. No communication needed.

The sum is computed without either party revealing their raw salary.

(b) Boolean XOR sharing is more natural for AND gates.

Boolean sharing represents bits and XOR is free. AND uses Beaver triples.

Additive sharing can handle AND/multiplication too, but requires its own Beaver-style multiplication triple. For pure bit operations, Boolean is cleaner.

(c) Addition is linear in additive shares.

$$(a_1 + a_2) + (b_1 + b_2) = (a_1 + b_1) + (a_2 + b_2).$$

Each server adds its own shares: S_A computes a_1+b_1 , S_B computes a_2+b_2 . Summing the results gives $a+b$. No values cross party boundaries.

(d) Multiplication is nonlinear.

$$(a_1+a_2) \times (b_1+b_2) = a_1b_1 + a_1b_2 + a_2b_1 + a_2b_2.$$

Terms a_1b_2 and a_2b_1 require values from different servers. This is the same problem as AND in Boolean sharing — cross terms require interaction or preprocessed correlated randomness (multiplication triple).

Q47 — MPC Applications — Match Function to Use Case

The slides list these MPC-computable functions. For each, state which sharing type is most natural and give a real-world use case.

(a) $x > y$ (comparison)

(b) $(x_1 + x_2 + \dots + x_n)/n$ (average)

(c) $a \text{ AND } b$ (Boolean AND)

(d) $\text{ReLU}(x) = \max(0, x)$ (ML activation)

(e) AES by MPC

ANSWER — Q47

(a) $x > y$ — Boolean sharing, comparison circuit.

Use case: two employees compare salaries without revealing exact amounts.

The comparison is implemented as a Boolean circuit over the binary representation.

(b) Average — Additive sharing.

Use case: average exam score across students without revealing individual grades.

Each student shares their score additively. Servers sum the shares locally, then divide by n . No individual score is revealed.

(c) $a \text{ AND } b$ — Boolean XOR sharing + Beaver triple.

Use case: mutual matching (both parties like each other).

The exact protocol covered in Lab 10.

(d) $\text{ReLU}(x) = \max(0, x)$ — Arithmetic/additive sharing + comparison.

Use case: privacy-preserving neural network inference.

The model provider shares weights; the client shares input. Neither learns the other's private data during inference.

(e) AES by MPC — Boolean sharing over AES circuit.

Use case: distributed key management — the AES key is secret-shared across multiple servers. No single server ever holds the full key. Encryption or decryption is performed as a multiparty computation.

STANDARDISATION — AES / EAXprime / CMAC · Exercises

Exercise — Standardisation Knowledge

Q48 — AES Standardisation — Facts

- (a) What document standardised AES and when?
- (b) How many designs were submitted? How many became finalists?
- (c) What was the runner-up design and who created it?
- (d) What are the three AES key sizes and their corresponding round counts?

ANSWER — Q48

(a) FIPS PUB 197, issued 2001, approved 2002.

FIPS = Federal Information Processing Standard. Published by NIST.

(b) 15 designs submitted, 5 became finalists.

The five finalists received extensive public cryptanalysis before a winner was chosen.

(c) Runner-up: Serpent, created by Ross Anderson, Eli Biham, and Lars Knudsen.

Serpent was known for its conservative security margin (more rounds, larger safety buffer). It lost to Rijndael primarily on performance grounds. Lars Knudsen is at SDU Sønderborg.

(d) Key sizes and rounds:

AES-128: 128-bit key, 10 rounds

AES-192: 192-bit key, 12 rounds

AES-256: 256-bit key, 14 rounds

Block size is ALWAYS 128 bits regardless of key size.

Q49 — EAXprime — The Cautionary Example

(a) What was EAXprime and what standard did it appear in?

(b) Who broke it and when?

(c) What is the main lesson from EAXprime for cryptographic deployment?

(d) A developer says "we use a standardised scheme so we are safe." Correct this statement using EAXprime as evidence.

ANSWER — Q49

(a) EAXprime (EAX') was an authenticated-encryption-like scheme standardised in **ANSI C12.22** for transport of electricity-meter-based data over networks.

(b) Broken in 2012 by Minematsu, Lucks, Morita, and Iwata.

They discovered a vulnerability and warned against use of the protocol. Note: Morita is the lecturer — this is personally relevant to the course.

(c) Main lesson:

Standardisation is a review milestone, NOT a permanent security guarantee. A scheme can be standardised, widely deployed, and later found to be broken. Security requires: use reviewed standards + correct implementation + ongoing analysis + willingness to migrate when vulnerabilities are found.

(d) Correction:

"Standardised" means the scheme was reviewed and accepted at a point in time. EAXprime was standardised and deployed — and then broken. A better statement: "we use a currently-reviewed standardised scheme with correct implementation, and we monitor for new cryptanalytic results." Standards provide good starting-point confidence but require ongoing attention.

Q50 — CMAC — Purpose and Context

- (a) What does CMAC stand for and what does it do?
- (b) What document standardised CMAC, and when?
- (c) Where is CMAC used? Name two specific protocols.
- (d) What security property does CMAC provide? What does it NOT provide?
- (e) How does CMAC relate to HMAC?

ANSWER — Q50

(a) CMAC = Cipher-Based Message Authentication Code.

It uses a block cipher (typically AES) to produce a fixed-length authentication tag over a message. The tag can be verified by anyone holding the same secret key. If the message is modified, the tag verification fails.

(b) NIST SP 800-38B, standardised in 2005.

(c) Low-power and wireless devices: WiFi and Bluetooth.

CMAC is efficient on constrained hardware that cannot run complex MAC algorithms. It is also used in industrial control and IoT contexts.

(d) CMAC provides: integrity + authenticity.

It does NOT provide confidentiality — the message remains in plaintext. Anyone can read the message; only the key holder can verify the tag. To get both confidentiality and integrity, use AEAD (e.g. AES-GCM) instead.

(e) HMAC vs CMAC:

Both are MACs using a shared secret key. Both provide integrity and authenticity.

HMAC is hash-based (uses SHA-256, SHA-384, etc.).

CMAC is cipher-based (uses AES).

CMAC is preferred in constrained environments where AES hardware acceleration is available but hash functions are not. Both are standardised and widely used.

Q51 — Standardisation Principles — Short Answer

(a) Why do different vendors need compatible cryptographic implementations?

(b) A team invents a custom cipher because "no attacker knows our algorithm." What is the problem with this approach?

(c) List four sound principles for deploying cryptography in a real system.

(d) AES and EAXprime both went through standardisation. Compare their outcomes.

ANSWER — Q51

(a) Interoperability.

A server running one vendor's TLS must be able to communicate with a client running another vendor's TLS. Standards define exact parameters, encodings, and behaviours so that independent implementations produce compatible results.

(b) Security through obscurity is not security.

Kerckhoffs's principle: a cryptographic system should be secure even if everything about it except the key is public knowledge. Custom unpublished ciphers have not undergone peer review, formal analysis, or cryptanalysis by experts. When the algorithm is eventually reverse-engineered (and it will be), there are no published defences. "Our algorithm is secret" provides weak, fragile, unanalysed security.

(c) Four deployment principles:

1. Use currently-reviewed standardised constructions (not textbook or custom).
2. Protect secret and private keys (access control, secure storage, rotation).
3. Use correct parameters — modes (AEAD not ECB), nonces, padding (OAEP not raw).
4. Track cryptanalytic findings and be prepared to migrate when vulnerabilities emerge.

(d) AES vs EAXprime outcomes:

AES: 15 submissions → 5 finalists → global public cryptanalysis → Rijndael selected → standardised FIPS PUB 197 → 20+ years with no practical attack. Success story.

EAXprime: standardised in ANSI C12.22 → deployed in metering networks → broken in 2012 by academic researchers → warning published. Cautionary story.

Same lesson from both: standardisation + ongoing review = confidence. Standardisation alone is not enough.